



EDUCACIÓN
SECRETARÍA DE EDUCACIÓN PÚBLICA



TECNOLÓGICO
NACIONAL DE MÉXICO®



Instituto **Tecnológico**®
de Aguascalientes

Instituto Tecnológico de Aguascalientes

Ingeniería en Tecnologías de la Información y las Comunicaciones

Tecnologías Inalámbricas

12:00 p m

Evaluativo FINAL

Profesor: Ricardo Alejandro Rodríguez Jiménez

Equipo:

Ramírez Rodríguez Martín de Jesús 23151193

Santacruz Rodarte Cristian Alexis 22150108

Pedroza Pérez Emmanuel 23151202

Fecha: 5 de junio de 2026

Índice

Introducción	3
Operación	4
Código ArduinoIDE	5
Código para el servidor	18
Estructuras.....	25
Métodos de Conexión	25
Problemas y soluciones	25
Métodos de aseguramiento.....	26
Conclusiones	27
Enlace Github	28



Introducción

El presente proyecto desarrolla un sistema tipo arcade en el que un videojuego se integra con una ESP32, lectura NFC/RFID y un servidor central para gestionar el acceso y el puntaje de cada jugador. Se espera que, cuando una tarjeta sea detectada, la ESP consulta la base de datos para verificar si el usuario existe y, con ello, autorizar el inicio del juego. Durante la partida, los puntos obtenidos se envían nuevamente al servidor, donde se almacenan y actualizan mediante una API en JavaScript conectada a una base de datos SQL Server en Docker, permitiendo llevar un control centralizado del progreso de cada jugador.



Operación

Utilizando los conocimientos del uso de NFC/RFID y el uso de Wifi en la ESP32 se simuló un centro Arcade.

Se desarrolló un juego dentro de la ESP32, basado en el conocido juego "Deal or No Deal".

Para iniciar el juego, se escanea el tag NFC del jugador en el módulo NFC relacionado al ESP32.

La ESP32 consultaría la base de datos en búsqueda del ID obtenido con la tarjeta NFC.

Al obtener la respuesta de la base de datos, la ESP32 indicará al juego que puede iniciar.

Los puntos obtenidos tras haber jugado, se enviarán a la ESP

La ESP32 enviará los puntos obtenidos así como el ID, para que el servidor pueda procesarlos y actualizar el puntaje acumulado

Se habilitará un servidor en MacOS, que ejecutará un contenedor en Docker para el uso de bases de datos SQL de Microsoft.

Se ejecutará un servidor JS para capturar lo que el ESP32 envía a la red.

El servidor JS actualizará los datos del puntaje obtenido.

En caso de que el usuario NFC no estén registrados, el mismo servidor creará un usuario con datos genéricos

Estructuras

Metodos de conexion

Y metodos de aseguramiento

Código ArduinoIDE

Librerías para el funcionamiento del código

```
#include <WiFi.h>

#include <HTTPClient.h>

#include <ArduinoJson.h>

#include <Wire.h>

#include <Adafruit_PN532.h>

#include <Arduino.h>
```

Variables para la conexión de la red y las rutas que buscare en el servidor API

```
const char* ssid = "TP-Link_3262";

const char* password = "99428167";

const char* testUrl = "http://192.168.0.104:3000/api/test";

const char* insertUrl = "http://192.168.0.104:3000/api/puntuaciones";
```

Pines para que funcione el NFC en I2C

```
#define SDA_PIN 21

#define SCL_PIN 22

Adafruit_PN532 nfc(-1, -1);
```

Variables para el estado del sistema y el funcionamiento del juego

```
bool sistemaListo = false;

int valores[7];

int ordenados[7];

bool abiertos[7];
```



```
int maletinPersonal;
```

```
int puntaje;
```

```
int oferta;
```

```
char usuario[50];
```

```
String uidStr = "";
```

```
bool juegoActivo = false;
```

```
static int etapa = 0;
```

```
static int descartados = 0;
```

Funciones del led integrado

```
#define LED_PIN 2
```

```
void setupLED() {
```

```
    pinMode(LED_PIN, OUTPUT);
```

```
}
```

Funcion que conecta a la red wifi con la contraseña y red definidas arriba. Tiene 20 intentos de conectarse si no lo consigue avisa de un error de conexion

```
void connectWiFi() {
```

```
    Serial.println("Conectando WiFi...");
```

```
    WiFi.mode(WIFI_STA);
```

```
    WiFi.begin(ssid, password);
```

```
int attempt = 0;
```

```
while (WiFi.status() != WL_CONNECTED && attempt < 20) {
```

```
    delay(500);
```



```

    Serial.print(".");

    attempt++;
}

if (WiFi.status() == WL_CONNECTED) {
    Serial.println("\nWiFi OK");
} else {
    Serial.println("\nWiFi FAIL");
}
}

```

Funcion que permite iniciar el juego una vez estando listo los sistemas de red, nfc y la conexión de base de datos

```

void iniciarJuego() {

    randomSeed(esp_random());

    for (int i = 0; i < 7; i++) {
        valores[i] = esp_random() % 1000 + 1;
        ordenados[i] = valores[i];
        abiertos[i] = false;
    }

    for (int i = 0; i < 6; i++) {
        for (int j = 0; j < 6 - i; j++) {
            if (ordenados[j] > ordenados[j + 1]) {

```



```

    int temp = ordenados[j];

    ordenados[j] = ordenados[j + 1];

    ordenados[j + 1] = temp;

}

}

}

```

```

Serial.println("\n=== DEAL OR NO DEAL ===");

Serial.println("Valores:");

for (int i = 0; i < 7; i++) {

    Serial.print(ordenados[i]);

    Serial.print(" ");

}

```

```

Serial.println("\nElige maletín (0-6)");

```

```

etapa = 0;

descartados = 0;

juegoActivo = true;

}

```

Método que recibirá los valores que se insertarán en la base de datos de servidor.

```

bool insertDataToDatabase(const char* name_disp, const char* usr, int score, const char* id_rfid)
{

    Serial.println("\nEnviando INSERT a MSSQL...");
}

```



Se crea un cliente HTTP y se especifica que ruta api se usara. Ademas se especifica que trabajara con objetos Json

```
HTTPClient http;  
  
http.begin(insertUrl);  
  
http.addHeader("Content-Type", "application/json");
```

Se crea el objeto Json con los atributos que se enviaran al insert

```
DynamicJsonDocument doc(200);  
  
doc["name_disp"] = name_disp;  
  
doc["usr"] = usr;  
  
doc["score"] = score;  
  
doc["id_rfid"] = id_rfid;
```

```
String jsonString;  
  
serializeJson(doc, jsonString);
```

Si los datos fueron enviados, se notifica, si hay algún error da el código de error

```
Serial.print("Datos enviados: ");  
  
Serial.println(jsonString);
```

```
int httpCode = http.POST(jsonString);
```

```
if (httpCode == 200) {  
  
    String response = http.getString();  
  
    Serial.println("✓ INSERT exitoso");  
  
    Serial.println(response);  
  
    http.end();
```



```
        return true;
    } else {
        Serial.print("X Error en INSERT: ");
        Serial.println(httpCode);
        String response = http.getString();
        Serial.println(response);
        http.end();
        return false;
    }
}
```

```
void setup() {
    Comunicacion serial
    Serial.begin(115200);
```

Activa el I2C usando los pines definidos

```
Wire.begin(SDA_PIN, SCL_PIN);
```

Inicia la function del LED integrado

```
setupLED();
```

Inicia la conexión wifi con el método connectWiFi

```
connectWiFi();
```



Inicia la conexión con la base de datos, el estado de la conexión se hace saber mediante la consola serial

```
if (WiFi.status() == WL_CONNECTED) {  
    sistemaListo = true;  
    Serial.println("Sistema listo ✓");  
} else {  
    Serial.println("Sistema NO listo ✗");  
    while (1) delay(1000);  
}
```

Habilita la funcionalidad NFC y lo configura para leer tags nfc

```
nfc.begin();
```

```
if (!nfc.getFirmwareVersion()) {  
    Serial.println("PN532 no detectado");  
    while (1);  
}
```

```
nfc.SAMConfig();
```

```
Serial.println("Acerca NFC para iniciar...");  
}
```

```
void loop() {
```

condicional para revisar que el sistema este funcionando antes de iniciar el juego

```
if (!sistemaListo) return;
```

Mientras el juego no este activo, espera un tag nfc,

```
if (!juegoActivo) {
```

```
    uint8_t uid[7];
```

```
    uint8_t uidLength;
```

```
    uidStr = "";
```

Cuando detecte un tag nfc, guarda los datos del UID del tag en un array

```
    if (nfc.readPassiveTargetID(PN532_MIFARE_ISO14443A, uid, &uidLength)) {
```

```
        Serial.println("NFC detectado -> iniciando juego");
```

```
        for (uint8_t i = 0; i < uidLength; i++) {
```

```
            if (uid[i] < 0x10) uidStr += "0";
```

```
            uidStr += String(uid[i], HEX;
```

```
        }
```

Se solicita un nombre de jugador

```
Serial.println("Escribe tu nombre de jugador:");
```

```
while (Serial.available() == 0) {
```

```
}
```

```
String nombre = Serial.readStringUntil('\n');
```

```
nombre.trim();
```

guarda el nombre en un array llamado usuario y le da la bienvenida

```
nombre.toCharArray(usuario, nombre.length() + 1);
```

```

Serial.print("Bienvenido, ");

Serial.println(nombre);

iniciarJuego();

}

```

```

return;

}

```

Funcionalidad del juego DEAL or no DEAL, en cada final del juego se llama la función para mandar los datos a la base de datos

```

if (Serial.available() > 0) {

    String entrada = Serial.readStringUntil('\n');

    entrada.trim();

    if (entrada.length() == 0) return;

    int opcion = entrada.toInt();

    if (etapa == 0) {

        if (opcion >= 0 && opcion < 7) {

            maletinPersonal = opcion;

            abiertos[maletinPersonal] = true;

            Serial.print("Has elegido el maletín ");

            Serial.println(maletinPersonal);

            etapa = 1;

            Serial.println("Descarta 3 maletines (0-6):");

        }
    }
}

```



```

} else if (etapa == 1) {
    if (opcion >= 0 && opcion < 7 && !abiertos[opcion]) {
        abiertos[opcion] = true;
        Serial.print("Maletín ");
        Serial.print(opcion);
        Serial.print(" abierto: $");
        Serial.println(valores[opcion]);
        descartados++;
    }
    if (descartados == 3) {
        int suma = 0, cuenta = 0;
        for (int i = 0; i < 7; i++) if (!abiertos[i]) { suma += valores[i]; cuenta++; }
        oferta = (suma / cuenta) * 0.7;
        Serial.print("Oferta del banco: $");
        Serial.println(oferta);
        Serial.println("¿Deal (1) o No Deal (0)?");
        etapa = 2;
    }
} else if (etapa == 2) {
    if (opcion == 1) {
        Serial.println("¡Aceptaste la oferta! Fin del juego.");
        puntaje = oferta;
        Serial.print("Tu puntaje es: ");
        Serial.println(puntaje);
    }
}

```

Manda los datos a la función para insertar a la base de datos con el nombre del juego, el nombre del jugador, puntaje y el UID del tag NFC y reinicia el esp



```

        if (insertDataToDatabase("DEAL OR NOT DEAL", usuario, puntaje, uidStr.c_str())) {
            Serial.println("\n✓✓✓ Puntaje subido a la base de datos✓✓✓");
        } else {
            Serial.println("\nX X X ERROR EN INSERT X X X");
        }
        delay(5000);

        ESP.restart();
    } else {
        Serial.println("No Deal. Descarta 2 maletines más:");

        descartados = 0;

        etapa = 3;
    }
} else if (etapa == 3) {
    if (opcion >= 0 && opcion < 7 && !abiertos[opcion]) {
        abiertos[opcion] = true;

        Serial.print("Maletín ");

        Serial.print(opcion);

        Serial.print(" abierto: $");

        Serial.println(valores[opcion]);

        descartados++;
    }
}

if (descartados == 2) {
    int suma = 0, cuenta = 0;

    for (int i = 0; i < 7; i++) if (!abiertos[i]) { suma += valores[i]; cuenta++; }

    oferta = (suma / cuenta) * 0.9;

    Serial.print("Oferta del banco: $");
}

```



```

Serial.println(oferta);

Serial.println("¿Deal (1) o No Deal (0)?");

etapa = 4;

}

} else if (etapa == 4) {

    if (opcion == 1) {

        Serial.println("¡Aceptaste la oferta! Fin del juego.");

        puntaje = oferta;

        Serial.print("Tu puntaje es: ");

        Serial.println(puntaje);

```

Manda los datos a la función para insertar a la base de datos con el nombre del juego, el nombre del jugador, puntaje y el UID del tag NFC y reinicia el esp

```

    if (insertDataToDatabase("DEAL OR NOT DEAL", usuario, puntaje, uidStr.c_str())) {

        Serial.println("\n✓✓✓ Puntaje subido a la base de datos✓✓✓");

    } else {

        Serial.println("\nX X X ERROR EN INSERT X X X");

    }

    delay(5000);

    ESP.restart();

} else {

    Serial.print("No Deal. Tu maletín personal era: $");

    Serial.println(valores[maletinPersonal]);

    puntaje = valores[maletinPersonal];

    Serial.print("Tu puntaje es: ");

    Serial.println(puntaje);

    Serial.println("Fin del juego.");

```



Manda los datos a la función para insertar a la base de datos con el nombre del juego, el nombre del jugador, puntaje y el UID del tag NFC y reinicia el esp

```
if (insertDataToDatabase("DEAL OR NOT DEAL", usuario, puntaje, uidStr.c_str())) {  
    Serial.println("\n✓✓✓ Puntaje subido a la base de datos✓✓✓");  
} else {  
    Serial.println("\nX X X ERROR EN INSERT X X X");  
}  
delay(5000);  
    ESP.restart();  
}  
}  
}  
}
```



Código para el servidor

Librerías para el funcionamiento del servidor

```
import express from 'express';  
import cors from 'cors';  
import bodyParser from 'body-parser';  
import sql from 'mssql';
```

Configuración de una aplicación express y aceptar peticiones json

```
const app = express();  
app.use(cors());  
app.use(bodyParser.json());
```

Credenciales necesarias para conectarse a la base de datos, la dirección IP del dispositivo donde se encuentre la base de datos y el certificado para conexiones locales

```
const dbConfig = {  
  user: 'sa',  
  password: 'CONTRA453N1!4', // "JasonMakana@2005", // Tu contraseña de Docker  
  server: '192.168.0.104',    // Tu propia máquina local  
  database: 'arcade_db',     // La base de datos del proyecto  
  port: 1433,                // Puerto nativo de SQL Server mapeado en Docker  
  options: {  
    encrypt: false,  
    trustServerCertificate: true  
  }  
};
```



Variable que guarda el ultimo jugador escaneado

```
let ultimoEscaneo = {  
  id_rfid: "-----",  
  usr: "Esperando...",  
  score: 0,  
  name_disp: "Ninguno"  
};
```

Ruta que recibirá las variables mandadas por el esp y checa si existe el jugador, si no existe crea un nuevo registro y si existe actualiza el registro

```
app.post('/api/puntuaciones', async (req, res) => {  
  const { name_disp, usr, score, id_rfid } = req.body;  
  
  console.log(`\n[API POST] Procesando puntos -> RFID: ${id_rfid} | Puntos a reportar:  
${score}`);
```

```
  try {  
    let pool = await sql.connect(dbConfig);  
  
    // Buscar si el usuario ya tiene un registro previo en la base de datos con esa tarjeta  
    let resultado = await pool.request()  
      .input('id_rfid', sql.VarChar, id_rfid)  
      .query(`SELECT SCORE FROM puntuaciones WHERE ID_RFID = @id_rfid`);  
  
    let nuevoTotal = score;
```

Si el jugador ya existe se suman los puntos

```

if (resultado.recordset.length > 0) {

    // EL JUGADOR YA EXISTE: Sumamos sus nuevos puntos al acumulado histórico

    let puntosActuales = resultado.recordset[0].SCORE;

    nuevoTotal = puntosActuales + score;

    await pool.request()

        .input('nuevo_score', sql.Int, nuevoTotal)

        .input('name_disp', sql.VarChar, name_disp)

        .input('usr', sql.VarChar, usr)

        .input('id_rfid', sql.VarChar, id_rfid)

        .query(`UPDATE puntuaciones

                SET SCORE = @nuevo_score, NAME_DISP = @name_disp, USR = @usr,

LAST_GAME = GETDATE()

                WHERE ID_RFID = @id_rfid`);

    console.log(`[SQL] ¡Puntos acumulados con éxito! Total anterior: ${puntosActuales} |

Nuevo Total Global: ${nuevoTotal}`);

} else {

    Si el jugador no existe se crea el registro con los atributos estipulados

    // JUGADOR NUEVO: Creamos su primera fila en la tabla con sus puntos iniciales

    await pool.request()

        .input('name_disp', sql.VarChar, name_disp)

        .input('usr', sql.VarChar, usr)

        .input('score', sql.Int, score)

        .input('id_rfid', sql.VarChar, id_rfid)

```



```

        .query(`INSERT INTO puntuaciones (NAME_DISP, USR, SCORE, ID_RFID)
                VALUES (@name_disp, @usr, @score, @id_rfid)`);

    console.log("[SQL] ¡Jugador nuevo registrado con éxito en el monedero global!");
}

// --- CORRECCIÓN CRUCIAL DE ARRANQUE AUTOMÁTICO ---
// Al terminar la partida, dejamos el buzón de login vacío (en líneas de guiones).
// De esta forma, Python guardará el score pero NO se reiniciará solo; se verá
// forzado a quedarse en la pantalla de espera hasta que la ESP32 mande un nuevo GET.
ultimoEscaneo = {
    id_rfid: "-----",
    usr: "Esperando...",
    score: 0,
    name_disp: "Ninguno"
};

res.status(200).json({
    status: "success",
    action: resultado.recordset.length > 0 ? "update" : "insert",
    total_score: nuevoTotal
});

} catch (err) {

    console.error("[ERROR SQL POST]", err.message);

    res.status(500).json({ status: "error", message: err.message });
}

```

```
}  
});
```

Ruta que recupera el score de una tarjeta RFID

```
app.get('/api/puntuaciones/:rfid', async (req, res) => {  
  const rfidConsultado = req.params.rfid;  
  console.log(`\n[API GET] Consultando puntos del RFID: ${rfidConsultado}`);
```

Si existe la tarjeta en el registro manda su puntaje

```
  try {  
    let pool = await sql.connect(dbConfig);  
    let resultado = await pool.request()  
      .input('id_rfid', sql.VarChar, rfidConsultado)  
      .query(`SELECT SCORE, USR FROM puntuaciones WHERE ID_RFID = @id_rfid`);  
  
    if (resultado.recordset.length > 0) {  
      console.log(`[SQL] Tarjeta encontrada. Saldo actual: ${resultado.recordset[0].SCORE}  
pts.`);
```

```
    // Llenamos el buzón para que Python sepa que un usuario acaba de loggarse
```

```
    ultimoEscaneo = {  
      id_rfid: rfidConsultado,  
      usr: resultado.recordset[0].USR,  
      score: resultado.recordset[0].SCORE,  
      name_disp: "Arcade_ESP32_WiFi"  
    };
```

```

res.status(200).json({
  existe: true,
  score: resultado.recordset[0].SCORE,
  usr: resultado.recordset[0].USR
});

```

Si no encuentra la tarjeta se crea un nuevo registro con 0 puntos

```

} else {
  console.log(`[SQL] Tarjeta nueva. Iniciando cuenta en 0 pts.`);

  // Llenamos el buzón con datos en 0 para usuario nuevo
  ultimoEscaneo = {
    id_rfid: rfidConsultado,
    usr: `User_${rfidConsultado.substring(0,4)}`,
    score: 0,
    name_disp: "Arcade_ESP32_WiFi"
  };

```

```

res.status(200).json({
  existe: false,
  score: 0,
  usr: `User_${rfidConsultado.substring(0,4)}`
});

```

```

}

```

```

} catch (err) {

  console.error("[ERROR SQL GET]", err.message);

```



```

        res.status(500).json({ error: err.message });
    }
});

```

Ruta que recupera el ultimo juego que jugo el usuario

```

app.get('/api/ultimo-movimiento', (req, res) => {
    // Enviamos el estado actual del buzón a Python
    res.status(200).json(ultimoEscaneo);

    // Vaciamos el buzón inmediatamente después de ser leído para que Python no lea doble
    ultimoEscaneo = {
        id_rfid: "-----",
        usr: "Esperando...",
        score: 0,
        name_disp: "Ninguno"
    };
});

```

```

// =====
// 4. ARRANQUE DEL SERVIDOR PUENTE IoT
// =====

const PORT = 3000;

app.listen(PORT, () => {
    console.log(`=====
=====`);

    console.log(` Servidor Monedero Arcade Centralizado corriendo en puerto ${PORT}`);

```



```
console.log(` Listo para acumular puntos por RFID y sincronizar con tus compañeros`);  
  
console.log(`=====
```


====`);
});

Estructuras

Este proyecto tuvo esta forma:

- ESP32 como dispositivo lector y agente de envío de datos.
- El juego como módulo recibe la autorización y devuelve el puntaje.
- El servidor JS como intermediario entre la ESP y la base de datos.
- La base de datos como almacenamiento de usuarios y scores.

Métodos de Conexión

- Conexión WiFi de la ESP32 al servidor, donde interviene la comunicación de un router que haya funcionado adecuadamente.
- Lectura de tarjeta NFC/RFID para obtener el ID.
- Envío de datos por HTTP POST/GET hacia el servidor.
- Conexión del servidor JS al otro mediante la librería de base de datos.
- Intercambio de información en formato JSON.

Problemas y soluciones

Entre los problemas encontrados:

- La conexión de la base de datos no funcionaba por la configuración del servidor.
- Cambiar rutas cada vez que se modificaba la base de datos y se reiniciaba.
- Configurar el servidor con la base de datos para que el grupo pueda utilizarlo.

Para solucionar

- Poner la información exacta de los atributos de la base de datos en el servidor y esp32.
- Tener una variable que contenga la ruta para solo modificar una línea de código.

Métodos de aseguramiento

- Verificar que la ESP esté conectada a WiFi antes de iniciar.
- Comprobar que el lector NFC detecte correctamente la tarjeta.
- Confirmar que el servidor responda antes de aceptar un envío.
- Validar si el usuario existe antes de actualizar el score.
- Crear un usuario genérico si la tarjeta no está registrada.
- Reiniciar el sistema o el juego después de completar la partida para evitar estados incorrectos.



Conclusiones

Martín de Jesús Ramírez Rodríguez:

Este sistema demostró cómo pueden integrarse hardware, software y bases de datos para crear una experiencia de juego funcional y organizada. La ESP32 actúa como puente entre la identificación del usuario, el videojuego y el servidor, mientras que la API se encarga de registrar, actualizar o crear usuarios de forma automática según sea necesario. Con ello, se logra una solución escalable para un centro arcade, capaz de gestionar partidas, puntajes e identificación de jugadores de manera eficiente.

Pedroza Pérez Emmanuel:

En este proyecto integrador, aplicamos todo el conocimiento adquirido en el semestre que nos ejemplifica las diferentes funcionalidades que podemos aplicar a las tecnologías inalámbricas. Usando las practicas anteriores de base logramos realizar un sistema de puntaje para juegos totalmente independientes con esta base podemos extrapolar las funciones de la práctica para tener sistemas de monitoreo de datos, ganado, clima etc.

Cristian Alexis Santacruz Rodarte:

Con el proyecto final se hizo uso de lo aprendido en la materia, haciendo uso de ESP32 se hizo una conexión hacia un servidor con base de datos para registrar puntuación dentro del videojuego creado, el proyecto nos hizo resaltar las habilidades de conexión aprendidas en la materia, haciendo uso de circuitos, ESP32, programación y conocimientos en bases de datos.



Enlace Github

A continuación, se adjunta el enlace con el código ArduinoIDE expuesto, así como el video de evidencia y documentación:

<https://github.com/MartinITAgs/WIFI-ESP32>

